

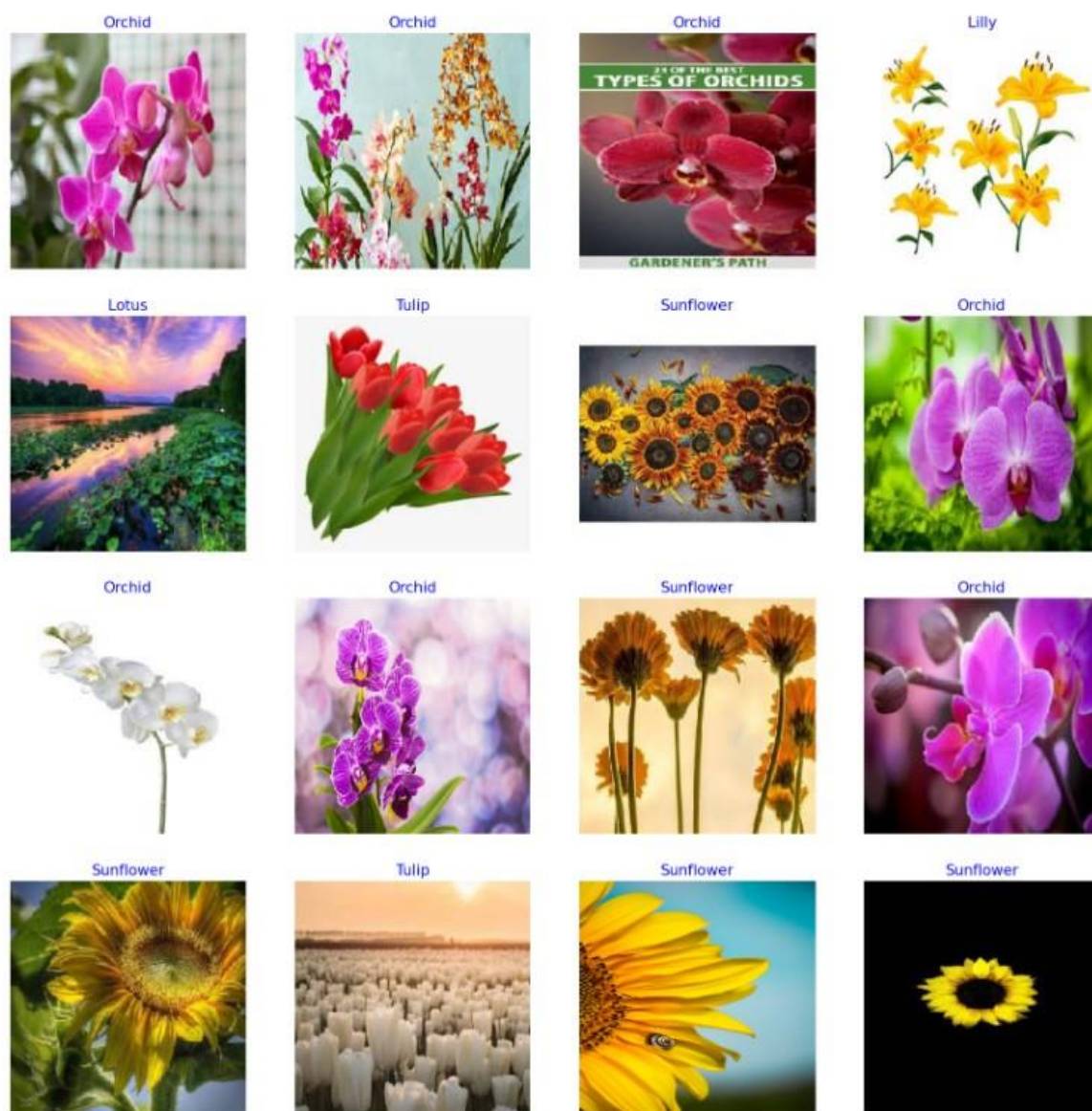
Strojno učenje – projektni zadatak

1. Uvod

1.1. Opis zadatka

Izraditi i istrenirati neuronsku mrežu kako bi se vršila klasifikacija određenog podatkovnog skupa. Podatkovni skup zove se „5 Flower Types Classification Dataset“, a sastoji se od 5 mapa sa po 1000 različitih slika za svaku od 5 vrsta cvjetova: ljiljana, lotusa, orhideje, suncokreta i tulipana.

Primjer slika iz podatkovnog skupa dan je slijedećom slikom:



Slika 1: primjer slika iz podatkovnog skupa

2. Odabir arhitekture mreže

Korištena su 4 različita pristupa rješavanju zadatka. Najprije je napravljen vlastiti model, a zatim mreže koje koriste već postojeće modele: DenseNet169/DenseNet121, EfficientNetB0/EfficientNetB1/EfficientNetB3 te VGG16.

Najbolji rezultati dobiveni su korištenjem EfficientNetB3 modela (maksimalna točnost na valid. skupu: 97,8%), a najgori na mreži sa VGG16 modelom (maksimalna točnost na valid. skupu: 89,6%).

Također, u svim slučajevima najbolji rezultati dobivani su pri korištenju batch size-a 32 naspram korištenja veličina 16 ili 64.

Budući da su najbolji rezultati dobiveni pomoću EfficientNetB3 modela, ta mreža je odabrana kao finalna te će se nju opisati u daljnjem tekstu.

2.1. Biblioteke i moduli

Za izradu mreže korištene su slijedeće biblioteke i moduli:

```
import os
import cv2
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
from sklearn.model_selection import train_test_split
import tensorflow as tf
from tensorflow import keras

# tensorflow i keras
from keras import Sequential
from keras._tf_keras.keras.preprocessing.image import ImageDataGenerator
from keras._tf_keras.keras.preprocessing.image import img_to_array, load_img

# micanje warninga
import warnings
warnings.filterwarnings("ignore")
```

```
from keras._tf_keras.keras.applications import DenseNet121, DenseNet169
from keras._tf_keras.keras.layers import *
from keras._tf_keras.keras.applications import ResNet50, Xception
from keras import regularizers
from keras._tf_keras.keras.callbacks import EarlyStopping, ReduceLROnPlateau
from keras._tf_keras.keras.regularizers import l2
```

```
from keras._tf_keras.keras.losses import BinaryCrossentropy
from keras._tf_keras.keras.initializers import he_normal
from keras._tf_keras.keras.optimizers import Adam, Adamax
```

2.2. Priprema podataka

Podaci se spremaju u *data_dir* direktorij iz baze koja sadrži slike cvijeća podijeljene u 5 različitih mapa.

Kreiraju su dvije liste: *filepaths* lista za spremanje putanje do svake datoteke te *labels* lista za spremanje pripadajuće oznake svake datoteke.

Linija `folders = os.listdir(data_dir)` izlistava sve mape u direktoriju nakon čega se prolazi kroz for petlju koja:

- konstruira putanju do svake mape
- izlistava sve datoteke u svakoj mapi
- pridjeljuje putanju i pripadajuću oznaku za svaku datoteku u listi (`filepaths.append(fpath)`, `labels.append(fold)`)

```
data_dir = "C:/Users/Emzzz/Desktop/su_KV/baza/flower_images"
filepaths = []
labels = []

folders = os.listdir(data_dir)
for fold in folders:
    foldpath = os.path.join(data_dir, fold)
    filelist = os.listdir(foldpath)
    for file in filelist:
        fpath = os.path.join(foldpath, file)
        filepaths.append(fpath)
        labels.append(fold)
```

2.3. Kreiranje DataFrame-a

Pozivanjem metode `pd.Series(predana_lista, naziv_novog_niza)` stvorene se liste pretvaraju u *Pandas* nizove a pozivanjem metode `pd.Concat([novostvoreni_niz1, novostvoreni_niz2], stupci)` ti se nizovi ulančavaju u jedan *DataFrame* sa putanjama i oznakama datoteka.

```
file_path_series = pd.Series(filepaths, name= 'filepaths')
label_path_series = pd.Series(labels, name='labels')
df = pd.concat([file_path_series, label_path_series], axis= 1)
```

2.4. Podjela skupa podataka na train, validation i test dijelove

80% skupa podataka dijeli se na skup za treniranje a 20% na tzv. „dummy“ odnosno pomoćni skup. Zatim se taj pomoćni skup dijeli na 50% za validaciju i 50% za testiranje odnosno 10% ukupnog podatkovnog skupa ide na validacijski, a 10% na testni skup, ostalih 80% na trening skup. *Shuffle* parametar je postavljen na true što osigurava da se podaci nasumično podijele u podskupove, a *random_state* parametar je postavljen na 42 kako bi se svaki put jednako slučajno dijelili skupovi.

```
train_df , dummy_df = train_test_split(df ,train_size = 0.8 , shuffle = True
,random_state = 42 )

valid_df , test_df = train_test_split(dummy_df ,test_size= 0.5 , shuffle =
True ,random_state = 42)

print(f"The shape of The Train data is: {train_df.shape}")
print(f"The shape of The Validation data is: {valid_df.shape}")
print(f"The shape of The Test data is: {test_df.shape}")
```

2.5. Augmentacija podataka i Image Data Generator metoda

Koristi se `ImageDataGenerator()` metoda za sva tri skupa (trening, validacijski i testni) te se sve slike unutar skupa skaliraju sa 1/255 kako bi se vrijednosti piksela postavile u interval [0,1].

Za trening skup također se vrši i: rotiranje podataka za 20 stupnjeva u bilo kojem smjeru, horizontalno i vertikalno rotiranje slike za 20%, shear transformacija odnosno stezanje slike za 0.2 radijana, zumiranje ili odzumiranje slike za 20%, horizontalno zrcaljenje slike, te popunjavanje piksela koji su možda izgubljeni promjenama u slici.

Sve te promjene rade se kako bi se trening skup proširio – kako bi se dobilo više ulaznih podataka budući da je sam po sebi ulazni dataset možda premalen.

```
train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest'
)

validation_datagen = ImageDataGenerator(rescale=1./255)
test_datagen = ImageDataGenerator(rescale=1./255)
```

2.6. Kreiranje podatkovnih generatora

Za sva tri skupa (trening, validacija i test) prave se podatkovni generatori koji će se po 32 slike predavati u svakom *batch*-u. Prije toga slike se postavljaju na jedinstvenu veličinu 240x240 piksela. Vrsta oznaka je kategorička.

```
batch_size = 32

print("Training Data")
train_ds = train_datagen.flow_from_dataframe(
    train_df,
    x_col = 'filepaths',
    y_col = 'labels',
    target_size = (224, 224),
    batch_size = batch_size,
    class_mode = 'categorical'
)

print("Validation Data")
valid_ds = validation_datagen.flow_from_dataframe(
    valid_df,
    x_col = 'filepaths',
    y_col = 'labels',
    target_size = (224, 224),
    batch_size = batch_size,
    class_mode = 'categorical'
)

print("Test Data")
test_ds = test_datagen.flow_from_dataframe(
    test_df,
    x_col = 'filepaths',
    y_col = 'labels',
    target_size = (224, 224),
    batch_size = batch_size,
    class_mode = 'categorical'
)
```

2.6. Brojanje oznaka

Linija `count = train_df['labels'].value_counts()` prebrojava pojavu svake od mogućih 5 klasa cvijeća u trening skupu.

Linija `class_indices_train = train_ds.class_indices` pridjeljuje svakoj od mogućih 5 klasa cvijeća jedan indeks pa je rezultat: Indeksi za trening skup:

```
{'Lilly': 0, 'Lotus': 1, 'Orchid': 2, 'Sunflower': 3, 'Tulip': 4}
```

```
count = train_df['labels'].value_counts()

class_indices_train = train_ds.class_indices
print("Indeksi za trening skup:", class_indices_train)
```

2.7. Izgradnja modela

Kao bazni model koristi se prethodno utrenirani model iz *EfficientNet* obitelji: *EfficientNetB3* (dobre rezultate davali su i B0, B1, ali B3 najbolje).

Model je sekvencijalan, a uz bazni sloj dodaje mu se i 3 *Dense* sloja sa 128, zatim 265 te 5 neurona. Zadnji *Dense* sloj ima 5 neurona jer ima 5 mogućih izlaznih klasa cvijeća.

Korišteni su `Flatten()` metoda koja pretvara podatke u jednodimenzionalno polje za prenošenje u slijedeći sloj i `Dropout()` regularizator koji se poziva dvaput (jednom sa 20% zanemarenih čvorova, a drugi put sa 50%).

Prva dva *Dense* sloja koriste „relu“ aktivacijsku funkciju, a zadnji naravno „softmax“.

Kao optimizator se koristi *Adamax* koji u ovom slučaju daje bolje rezultate nego *Adam*. Parametar `learning_rate` postavljen je na 0.001 – malen parametar znači rjeđe ažuriranje težina u modelu tijekom treniranja što u ovom slučaju daje bolje rezultate nego veći `learning_rate` npr. 0.01 ili 0.05.

`loss="categorical_crossentropy"` uspoređuje *one-hot* enkodirane oznake klase sa predikcijom klase te na taj način računa gubitke u mreži. Funkcija gubitka penalizira netočne predikcije (one koje su „daleko“ od stvarne vrijednosti klase).

Točnost je omjer točno predviđenih instanci i ukupnog broja instanci te u ovom zadatku daje najbolju mjeru toga koliko model dobro radi. Za realnu procjenu promatra se točnost na validacijskom skupu, a ne na trening skupu (budući da model može overfittati pa imati jako dobru točnost na trening, ali znatno slabiju na validacijskom i testnom skupu).

Dodavanje dodatnih *Dense* slojeva, kao i dodavanje *CNN* slojeva u vlastitom modelu nije značajno poboljšalo rezultate (u nekim slučajevima ih je i pogoršalo), ali je dosta produžilo vrijeme treniranja. Iz tog razloga je arhitektura zadržana na 7 slojeva što se može vidjeti i pozivanjem `model.summary()` metode (pri tome se *EfficientNet* ustvari sastoji od 237 slojeva ali se promatra kao 1 blok radi jednostavnosti).

```
base_model=tf.keras.applications.efficientnet.EfficientNetB3(
    include_top=False,
    weights="imagenet",
    input_shape=(224,224,3))

model=Sequential([
```

```

base_model,
Flatten(),
Dense(128,activation="relu"),
Dropout(rate=0.2),
Dense(265,activation="relu"),
Dropout(rate=0.5),
Dense(5,activation="softmax")

])
model.compile(Adamax(learning_rate=0.001),loss="categorical_crossentropy",metrics=["accuracy"])
model.summary()

```

3. Proces treniranja mreže

3.1. Treniranje modela

Metoda `.fit()` pokreće treniranje modela. Kao parametre joj se predaje trening skup podataka, broj epoha, *shuffle* parametar postavljen na „false“ (što znači da se podaci ne „miješaju“ prije svake epohe, već uvijek ulaze istim redoslijedom u epohu) i validacijski skup podataka na kojemu se evaluira gubitak i točnost.

Povećavanjem broja epoha model ne daje bolje rezultate. Najveći isprobani broj epoha bio je 60 epoha, ali rezultati su bili jednaki kao i na manjem broju, a treniranje je trajalo gotovo 4 sata.

```

history = model.fit(train_ds, epochs= 15, shuffle = False,
#                 steps_per_epoch = len(train_ds),
#                 validation_data = valid_ds,
#                 validation_steps = len(validation_ds)
)

```

3.2. Spremanje rezultata

Stvara se *dictionary* sa linijom `history.history`, koji sadrži gubitke i točnost trening i validacijskog skupa u svim epohama.

Prolazio se kroz sve epohe i u „*results*“ *dictionary* se spremaju mjesta za epohe, gubitak na trening skupu, točnost na trening skupu, gubitak na valid. skupu te točnost na valid. skupu.

Pomoću *pickle* biblioteke otvara se „*training_history.pkl*“ datoteka u binarnom obliku te se iz nje sprema rezultate treniranja u „*results*“ *dictionary*.

Iz „*results*“ *dictionary*-ja se pomoću for petlje rezultati spremaju u tekstnu datoteku čija je putanja predana kao {*text_file_path*}.

```
history_dict = history.history

print(history_dict)

epochs = range(1, len(history_dict['loss']) + 1)
results = {
    'epoch': epochs,
    'loss': history_dict['loss'],
    'val_loss': history_dict['val_loss'],
    'accuracy': history_dict['accuracy'],
    'val_accuracy': history_dict['val_accuracy']
}

import pickle

with open('training_history.pkl', 'rb') as f:
    results = pickle.load(f)

text_file_path = "C:/Users/Emzzz/Desktop/su_KV/strojno-ucenje-lv-
ovi/training_history.txt"

with open(text_file_path, 'w') as text_file:
    for key, value in results.items():
        text_file.write(f'{key}: {value}\n')

print(f'Results saved to {text_file_path}')
```

4.Evaluacija mreže

Rezultati mreže prikazani su slijedećom slikom:

